

the unneeded bits. Go green. In other words, get started on Reversible Computing.

Going Ballistic...

To understand how we can do this, we need to understand “ballistic” circuits. Reversible Computing is not simple to explain, but the first step is the broad picture. For that, all you need to do is to think of computation as a physical process. Once you’re in “something-physical-is-going-on-in-the-processor” mode, instead of thinking “software-is-running-on-the-processor,” you’ll find the following easy enough to understand.

A ballistic physical process is one where the system moves forward propelled by its own momentum, without much of its own energy being converted into heat. Think of an ICBM—an intercontinental ballistic missile: it doesn’t heat up too much in the atmosphere. It “coasts” along once it’s been propelled into the atmosphere. Now think of a ballistic *circuit*, though it might be hard to imagine.

Since we’re talking physics now, one needs to see that an ICBM isn’t all that different from a circuit—they both follow the laws of physics! How ballistic a process can be is quantified by its “quality factor,” called *Q*. This is simply the ratio between the amount of energy involved in carrying out the process and the amount that gets dissipated as heat. The idea here is to construct computers with tiny, nanoscale oscillators with a high *Q*. (Oscillators are spring-like devices: what makes your CPU run at 3.2 GHz is an oscillator, which generates a recurring signal.) The nano-oscillators would be on the silicon, and would enable the chip to run in a ballistic way: they would capture most of the energy used in one calculation and then use it in another—at least in pure theory. One feeds in the initial data, gives the processor a kick-start, and it coasts (like an ICBM) from one step of the computation to the next. This means only a small fraction of its energy being lost to heat at each step.

The concept is slightly analogous to hybrid cars that take the energy generated during braking and convert it into electricity that can be used to power the car.

The biggest challenge facing reversible computing is probably that of designing oscillators that have a high enough *Q*. There is no fundamental reason that it cannot be done—we can leave that to circuit engineers!

...But Keeping Your Cool

Coming back to heat, a physical process that is highly ballistic (such as what we’ve described above) means that it is also adiabatic to a high degree. “Adiabatic” means “without flow of heat.” A field called adiabatic circuits studies logic circuits that operate mostly in adiabatic mode.

Link that to the fact that a physical process can be adiabatic only if it is logically reversible—we mean it should employ “reversible logic.” The logic we commonly use—such as “1 AND 0 = 0”, “1 OR 0 = 1”, and so on, are not reversible.



The entire long-term future of our civilisation will depend on exploring reversible computing

Dr Michael Frank
Junior Faculty Member
FAMU-FSU College of
Engineering

For example:

- ❑ When you do an AND operation on a 0 and 1, you get a 0
- ❑ You also get a 0 when you do an AND on a 0 and a 0

So if you have a 0, how do you know what the inputs were? That’s irreversible logic.

It turns out that we *can* have reversible logic—logic that doesn’t discard bits. We hope you realised that in the above example, the two bits that went in as inputs were lost; with reversible logic, they won’t.

Playing Pool

What we have thus far is that we need ballistic circuits; ballistic circuits are highly adiabatic; and adiabatic circuits should employ reversible logic. But before that, here’s another way of looking at what’s going on in Reversible Computing. We need to ask, what is a computation? You feed in initial data, do some operations, and whatever state the computer reaches is the result.

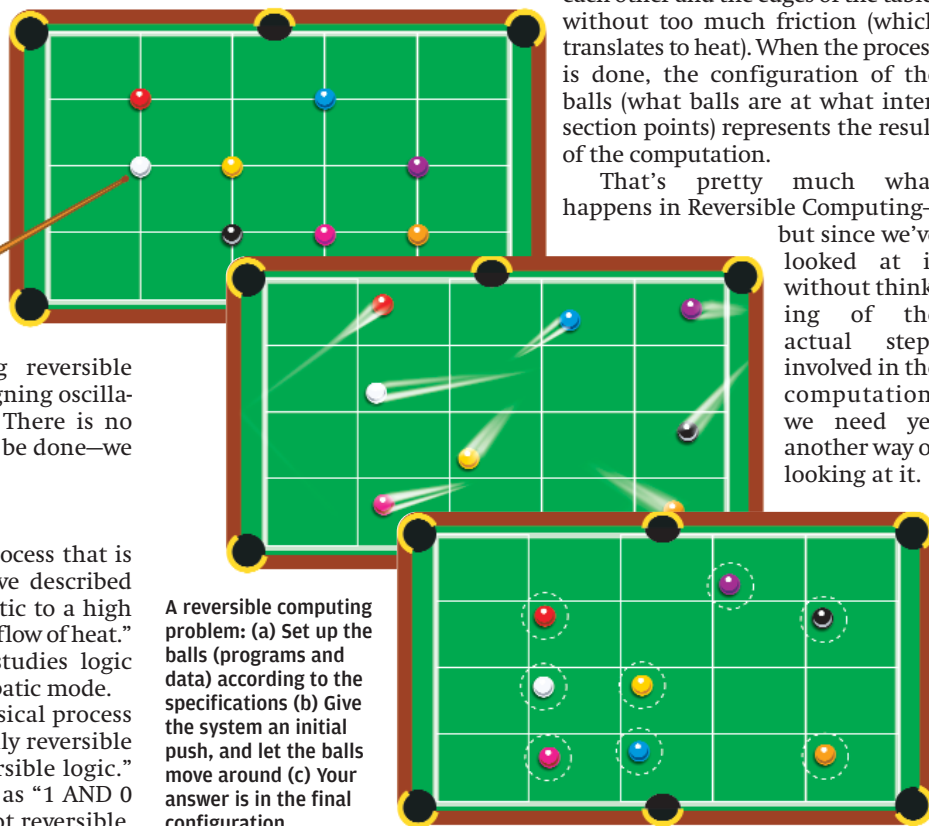
In a regular setup, this happens in a step-by-step, irreversible fashion. The system moves from one state to another, but it’s all in the forward direction. And this is the only way we currently think about computation.

Instead, think of a perfectly smooth pool table. Divide the table into a grid. At the intersections of the grid, the presence of a ball represents a 1; the absence of a ball is a 0.

Now, to describe a problem, you’ll need to figure how to configure the pool balls. Once that’s figured out, place the balls on the table, and then give the cue ball a slight push. Since it’s a pool table, what will ensue will be ballistic—the balls will move in all sorts of directions, bouncing off

each other and the edges of the table, without too much friction (which translates to heat). When the process is done, the configuration of the balls (what balls are at what intersection points) represents the result of the computation.

That’s pretty much what happens in Reversible Computing—but since we’ve looked at it without thinking of the actual steps involved in the computation, we need yet another way of looking at it.



A reversible computing problem: (a) Set up the balls (programs and data) according to the specifications (b) Give the system an initial push, and let the balls move around (c) Your answer is in the final configuration